

Phase 4: AI Engineering



Detailed Session Plans

Session 1: Applied AI & Application Development

Focus: Harnessing foundation models, cloud ecosystems, vendor SDKs, vector stores, agentic frameworks, and production patterns to rapidly build, secure, and evaluate end-to-end AI applications.

- **AI Building Blocks & Model Integration**
 - **Foundation Models & LLM APIs:** Multi-provider API integration using core SDKs (OpenAI, Google Gemini, Anthropic).
 - **Unified Model Gateway:** Abstracting provider dependencies using **LiteLLM** for dynamic routing, unified keys, retries, and automated fallback strategies.
 - **Prompt Engineering & Structured Outputs:** System prompt framing, few-shot techniques, context management, and enforcing JSON schemas/function calling using Pydantic and Instructor.
 - **Embeddings & Vector Databases:** Dense embeddings, similarity metrics, and vector store architectures (e.g., Pinecone, PGVector, Qdrant).
- **AI Development Ecosystems & Enterprise Cloud Platforms**
 - **Orchestration Libraries:** Building context-driven pipelines using **LangChain** and **LlamaIndex**.
 - **Agentic Frameworks:** Designing stateful agent workflows, multi-agent teams, and graph topologies using **OpenAI Agents SDK** and **LangGraph**.
 - **Cloud Ecosystems:** Evaluating Azure AI (Azure OpenAI, AI Foundry, Document Intelligence), AWS AI (Bedrock, Textract, Transcribe), and Google Vertex AI & Gemini. Framework for selecting tools based on cost, compliance, latency, and operational constraints.
- **Building Core Application Architectures**
 - **Architectural Patterns:** Designing Retrieval-Augmented Generation (RAG), Autonomous Agents, Chatbots & Copilots, Document Intelligence pipelines, and Workflow Automation engines.
 - **End-to-End System Topology:** Mapping low-coupling request flows connecting user interfaces, orchestration layers, database indexes, and model endpoints.
- **Productionizing, Guardrails, & Optimization**
 - **Rapid Deployment:** Exposing web services via **FastAPI** (REST, Server-Sent Events/SSE streaming) and interactive frontends with **Streamlit**.
 - **Guardrails, Safety, & Security:** Implementing prompt injection defense, jailbreak detection, output validation, PII masking (NeMo Guardrails / Guardrails AI), OAuth2/IAM, and API gateway rate-limiting.

- **Caching Paradigms:** Implementing **Prompt Caching** (provider-level KV cache reuse) vs. **Semantic Caching** (Redis / Vector DB similarity matching) to serve duplicate queries instantly.

Session 2: Engineering AI Systems & Hardware Scale

Focus: AI system design principles, industrial model serving infrastructure, hardware acceleration, quantization, distributed training/inference, and enterprise operational resilience.

- **AI System Architecture & Scalable Infrastructure**
 - **System Design Principles:** Architectural patterns for AI/ML systems; decoupling training pipelines from inference execution; stateless design paradigms.
 - **Processing Topologies:** Batch vs. real-time interactive processing; microservices architecture vs. event-driven systems.
 - **Resilience & Storage:** Scalability and High Availability (HA) patterns; multi-tier storage, caching strategies (Redis, vector stores), and message queues (RabbitMQ/Kafka).
- **Model Deployment, Serving, & Infrastructure**
 - **Model Serving Engines:** Deploying trained weights using specialized industrial inference servers (**vLLM**, **Triton Inference Server**, **TorchServe**).
 - **Interface Protocols:** High-concurrency REST APIs vs. low-overhead binary **gRPC** protocols (Protocol Buffers) for microservice communication.
 - **Resource Management:** GPU scheduling, VRAM boundary allocation, multi-model serving on shared hardware, autoscaling, and dynamic load balancing.
- **Performance Optimization, Acceleration, & Hardware Scale**
 - **GPU Architecture & Memory Optimization:** Managing HBM memory bandwidth vs. SRAM compute; analyzing latency vs. throughput trade-offs.
 - **Graph Compilation & Execution Runtimes:** Exporting computational graphs to **ONNX Runtime** and Ahead-Of-Time (AOT) compilation using **NVIDIA TensorRT**.
 - **Quantization Paradigms:** Precision downscaling across **FP16**, **BF16**, **INT8**, and **FP8** using post-training quantization (PTQ) and calibration datasets.
 - **High-Throughput Generation:** Dynamic server-side batching, **Continuous Batching**, Key-Value (KV) cache optimization, and **Speculative Decoding** for generation acceleration.
- **Distributed AI, Training, & Production Operations**
 - **Distributed Architectures:** Multi-GPU and multi-node compute topologies; **PyTorch Distributed Data Parallel (DDP)** and model parallelism techniques.
 - **Observability & Health Probes:** Full-stack observability (system metrics, structured JSON logging, distributed tracing); liveness and readiness probes (`/healthz`).
 - **Benchmarking & FinOps:** Performance profiling (Time to First Token / TTFT, Inter-Token Latency / ITL, requests per second), reliability/fault tolerance, and cost optimization & capacity planning.



High-Level Roadmap

1. **Access & Orchestrate (Session 1):** Connect to foundation models using LiteLLM, structure outputs with Pydantic, and build agentic workflows with LangGraph/LangChain.
2. **Secure & Monitor (Session 1):** Apply input/output guardrails, implement semantic/prompt caching for cost reduction, and instrument full-stack tracing with Langfuse.
3. **Architect & Deploy (Session 2):** Design decoupled microservices using gRPC and message queues, deploying trained models on industrial servers like Triton, TorchServe, or vLLM.
4. **Compile & Quantize (Session 2):** Convert dynamic deep learning graphs into ONNX/TensorRT engines and downscale weights (FP16/BF16/INT8/FP8) for maximum hardware efficiency.
5. **Scale & Optimize (Session 2):** Leverage PagedAttention, KV Caching, Speculative Decoding, and PyTorch DDP for multi-GPU/multi-node scale while profiling SLAs (TTFT, ITL).

s



Learning Objectives (Bloom's Taxonomy)

By the end of this module, all learners will be able to:

1. **Develop** multi-provider AI applications using unified routers (LiteLLM), structured output schemas (Pydantic/Instructor), and agentic workflows (LangGraph/LangChain). (*Bloom's: Apply*)
2. **Implement** multi-tier caching (Prompt & Semantic Caching), security guardrails, and full-stack tracing (Langfuse) to protect and optimize live AI endpoints. (*Bloom's: Apply*)
3. **Deploy** deep learning models using industrial serving engines (**vLLM**, **Triton**, **TorchServe**) with gRPC/REST APIs and GPU resource scheduling. (*Bloom's: Apply*)
4. **Compile** dynamic PyTorch models into optimized ONNX and NVIDIA TensorRT engines with multi-precision quantization (**FP16**, **BF16**, **INT8**, **FP8**). (*Bloom's: Analyze*)
5. **Configure** continuous batching, KV caching, and speculative decoding to maximize hardware throughput and minimize generation latency. (*Bloom's: Apply*)
6. **Profile** distributed AI architectures using PyTorch DDP across multi-GPU setups while monitoring system health, TTFT, and ITL metrics. (*Bloom's: Evaluate*)

By the end of the lesson, some students will be able to:

1. **Architect** multi-cloud enterprise solutions balancing Azure AI, AWS Bedrock, and Google Vertex AI based on compliance, latency, and cost trade-offs. (*Bloom's: Design*)

2. **Optimize** multi-node distributed training and Tensor Parallelism topologies to balance compute-bound vs. memory-bound bottlenecks under high-concurrency SLAs. (*Bloom's: Create*)

Prerequisites

Students are assumed to have the following prior knowledge:

1. **Python Engineering:** Advanced OOP, context managers, generators, and familiarity with concurrent/asyncio programming in Python.
2. **Machine Learning Fundamentals:** Solid understanding of embeddings, vector spaces, PyTorch tensors, and standard ML workflows.
3. **Linux & Web Basics:** Familiarity with REST APIs, JSON, environment variables (`.env`), HTTP status codes, and bash scripting.
4. **Cloud & Hardware Basics:** Understanding of API key management, basic client-server communication, and memory hierarchies (RAM vs. GPU VRAM).

Online Learning Roadmap & Resource Design

Recommended Reading & Documentation

- [Claude Learning Path](#)
- [Prompt Engineering Guide](#)
- [Gen AI for Beginners - Microsoft](#)
- [vLLM Core Architecture: PagedAttention Paper](#)
- [PyTorch Distributed Data Parallel \(DDP\) Tutorial](#)
- [OpenAI Production Best Practices](#)
- [Hugging Face Optimum \(ONNX & TensorRT\)](#)
- [Understanding async AI Systems](#)